



Responsive Design & Mini Project



Media Queries



CSS Units



Animations



Hover & Transitions

"Modern websites must work on all screen sizes."



What is Responsive Design?

Responsive design means a website **automatically adjusts** its layout and content to look great on any device — from a 4-inch phone to a 27-inch monitor.

Mobile

Single-column, thumb-friendly

Tablet

Two-column, touch-optimized

Laptop

Multi-column layouts

Desktop

Full-width, rich visuals

Why It Matters

- Better user experience across all devices
- Higher Google SEO rankings
- Professional, polished appearance

Classic Example

A desktop layout with three side-by-side columns **stacks vertically** on a mobile screen — content stays readable without horizontal scrolling.

Media Queries

Media queries are the backbone of responsive CSS. They apply styles **only when specific screen conditions are met** — like a screen width below 768px.

Real-World Example

```
.box {  
  width: 400px;  
  height: 200px;  
  background-color: blue;  
  color: white;  
  font-size: 30px;  
}  
  
/* Mobile */  
@media (max-width: 480px) {  
  .box {  
    width: 100%;  
    background-color: red;  
    font-size: 18px;  
  }  
}  
  
/* Tablet */  
@media (min-width: 481px) and (max-width: 768px) {  
  .box {  
    width: 80%;  
    background-color: green;  
  }  
}  
  
/* Laptop */  
@media (min-width: 769px) and (max-width: 1024px) {  
  .box {  
    background-color: orange;  
  }  
}  
  
/* Desktop */  
@media (min-width: 1025px) {  
  .box {  
    background-color: purple;  
  }  
}
```

Desktop: Row

Flex-direction set to row for wide screens



Mobile: Column

Media query switches flex-direction to column

The same container flows **horizontally on desktop** and **vertically on mobile** — no JavaScript needed.

Common Screen Sizes:

Device	Width Range
-----	-----
Mobile Small	0px to 480px
Mobile Large	481px to 767px
Tablet	768px to 1023px
Laptop	1024px to 1279px
Desktop	1280px and above

Building Mobile-Friendly Websites

Best Practices

→ Use Flexbox

Flexible, responsive layouts with minimal code

→ Use Percentages

Widths that scale relative to their parent

→ Apply Media Queries

Override styles at specific breakpoints

→ Avoid Fixed Widths

Never use `width: 900px` on full layouts

Common Mobile Pitfalls

⊗ Watch out for these issues on small screens

- **Text too small** — always use readable font sizes (min 16px)
- **Overflow content** — content spills outside the screen
- **Buttons too close** — hard to tap accurately
- **Horizontal scrolling** — a major UX red flag

CSS Units for Responsive Design

Choosing the **right CSS unit** is key to building layouts that adapt fluidly across screen sizes.



px — Pixels

Fixed size. Best for borders, icons, and small details.

```
width: 300px;
```



% — Percentage

Relative to the parent element. Great for fluid widths.

```
width: 50%;
```



vh — Viewport Height

A percentage of the screen's total height.

```
height: 100vh;
```



vw — Viewport Width

A percentage of the screen's total width.

```
width: 100vw;
```

 Use %, vh, and vw for responsive layouts. Reserve px for fine-tuned, fixed elements.

CSS Animations

CSS animations bring your UI to life — no JavaScript required. Define movement using `@keyframes` and apply it with the `animation` property.

Animation Example

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: blue;  
  animation: move 2s infinite;  
}  
  
@keyframes move {  
  0% { transform: translateX(0); }  
  100% { transform:  
    translateX(200px); }  
}
```

Key Properties

- `animation-name` — links to a `@keyframes` rule
- `animation-duration` — how long one cycle takes
- `animation-iteration-count` — use `infinite` to loop
- `transform: translateX()` — moves element horizontally

✓ Result: The box slides back and forth continuously across the screen.

Hover Effects

The `:hover` pseudo-class triggers styles when a user moves their cursor over an element — a simple way to add interactivity.

Button Hover

```
button {  
  background-color: blue;  
  color: white;  
  padding: 10px 20px;  
}  
  
button:hover {  
  background-color: red;  
}
```

Image Zoom Hover

```
img:hover {  
  transform: scale(1.1);  
}
```

Where to Apply Hover Effects



Buttons

Color or scale change on interaction



Images & Cards

Zoom, shadow, or overlay effects



Nav Links

Underline or color highlight on hover

CSS Transitions

Transitions make property changes **smooth and gradual** instead of instant. Add one line to any element to transform a jarring jump into a polished animation.

Without Transition

```
button:hover {  
  background-color: green;  
  /* Color snaps instantly */  
}
```

With Transition

```
button {  
  background-color: blue;  
  transition: 0.5s;  
}  
  
button:hover {  
  background-color: green;  
  /* Smoothly fades over 0.5s */  
}
```

Common Use Cases

Hover Effects

Smooth color & scale changes

Card Animations

Lift or shadow on hover

Menu Interactions

Slide-in and fade transitions

 `transition: 0.5s;` applies a 0.5-second ease to **all** changing properties. You can also target one property: `transition: background-color 0.3s ease;`

Responsive Landing Page — Putting It Together

Here's how a full hero section combines every concept from today into one clean, responsive component.

HTML Structure

```
<section class="hero">
  <div class="content">
    <h1>Learn Web Development</h1>
    <p>Start your coding journey today.</p>
    <button>Join Now</button>
  </div>
</section>
```

CSS

```
.hero {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  text-align: center;
}

@media screen and (max-width: 768px) {
  h1 { font-size: 24px; }
}
```

Concepts in Action

Flexbox

Centers content horizontally & vertically

vh Unit

Hero fills the full screen height

Media Query

Text shrinks gracefully on small screens

Responsive Text

Font size adapts to viewport



Practical Task & Homework

Today's Task

Build a Responsive Landing Page using everything from Day 5.

- Hero section with heading, description & button
- Background image using vh units
- Flexbox for centered layout
- Hover effect on the button with a transition
- Media query for mobile breakpoint

Homework Challenge

Extend your landing page to work across **all three breakpoints**:

01

Mobile

Responsive text sizes & stacked layout

02

Tablet

Two-column sections, proper spacing

03

Desktop

Full layout with hover animations



Bonus: Add a mobile navigation menu with smooth transitions.